DATA 412/612 · Statistical Programming in R · Week 14

14.2 Iteration: Loops and purrr

Pass the function in. map() walks a list, across() walks columns, and a for-loop is for the index.

Pass a function in, and let `map()` or `across()` walk the pieces. A for-loop is still the tool when you need the index, or when you do not know how many steps you will take.

By the end of this lecture you should be able to:

1. Tell a vector, a list, and a data frame apart, including `[` and `[[`.
2. Treat a function as an object you can pass to another function.
3. Repeat a function down columns with `across()`.
4. Walk a list or the columns of a data frame with `map()`.
5. Write a named function, or an anonymous one with `\(x)`.
6. Pick `map_dbl()`, `map_int()`, `map_chr()`, or `map_lgl()` from the type you want back.
7. Fit one model per group with `group_modify()` or with `split()` plus `map()`.
8. Walk two or more inputs with `map2()` and `pmap()`.
9. Use `walk()` when the point is a file or a plot, not a returned value.
0. Put a `map()` next to the for-loop it replaces.

Lecture 14.1 has the longer account of vectors and lists. The rowwise lecture has the longer account of `across()`.

---

1. A list, then a data frame

An atomic vector holds one type. A list can mix types, including a function. A data frame is a list of columns that all have the same length.

```
x <- list(greeting = "hello", number = 1, operation = mean)
x[1]
x[[1]]
x$greeting
```

```
## $greeting
## [1] "hello"
## [1] "hello"
## [1] "hello"
```

`x[1]` is still a list. `x[[1]]` and `x$greeting` are the element inside.

```
df <- tibble(
  name = c("Alice", "Bob", "Carol"),
  age = c(21, 25, 22),
  score = c(85, 90, 88)
)
df["age"]
df[["age"]]
```

```
## # A tibble: 3 × 1
##     age
##   <dbl>
## 1    21
## 2    25
## 3    22
## [1] 21 25 22
```

`df["age"]` is a one-column data frame. `df[["age"]]` is the numeric vector. `map()` can walk a data frame because each column is one element of that list.

---

2. A function is an object

`mean` is an object. `mean(1:10)` calls it. You can store the function under another name and call that name.

```
f <- mean
f(c(10, 20, 30))
```

```
## [1] 20
```

`map()` and `across()` take the function as an argument. Write `mean`, not `mean()`. The parentheses would call it immediately, before `map()` has an element to hand it.

---

3. A for-loop, pre-allocated

`seq_along(x)` is `1, 2, …, length(x)`. Prefer it to `1:length(x)`, which misbehaves on an empty `x`. Growing a vector with `c()` inside the loop is slow. Make the output first, then fill it.

```
x <- 1:5
cumvec <- vector("double", length(x))
total <- 0
for (i in seq_along(x)) {
  total <- total + x[[i]]
  cumvec[[i]] <- total
}
cumvec
```

```
## [1]  1  3  6 10 15
```

`vector("double", n)` is `n` zeros. You overwrite them. `x[[i]]` pulls one element. `cumvec[[i]] <-` writes one element back.

`while` when `n` is unknown

A `for` loop needs the sequence up front. A `while` loop keeps going until a condition is false:

```
nheads <- 0
flips <- 0
while (nheads < 3) {
  flips <- flips + 1
  if (rbinom(1, size = 1, prob = 0.5) == 1) {
    nheads <- nheads + 1
  }
}
```

Use `while` when you cannot say how many steps you need. Add `flips` as the last line when you want the count. That number changes from run to run.

---

4. `across()` on columns

`across()` is the dplyr form of "the same function, many columns." It lives inside `summarize()` or `mutate()`. The full treatment, including `rowwise()` and `c_across()`, is the rowwise lecture. Two patterns matter here.

One number per column, inside `summarize()`. Pass `mean` and `sd` as functions. `.names` builds names from the column and the function.

```
mtcars |>
  summarize(across(
    c(mpg, wt),
    list(mean = mean, sd = sd),
    .names = "{.col}_{.fn}"
  ))
```

```
##   mpg_mean   mpg_sd wt_mean     wt_sd
## 1 20.09062 6.026948 3.21725 0.9784574
```

`where(is.numeric)` selects by type. `is.numeric` has no parentheses. `where()` wants the function, not the result of calling it.

```
mtcars |>
  summarize(across(where(is.numeric), median))
```

```
##    mpg cyl  disp  hp  drat    wt  qsec vs am gear carb
## 1 19.2   6 196.3 123 3.695 3.325 17.71  0  0    4    2
```

Inside `mutate()`, a function that returns one number is repeated down the column. Every row of `x` below becomes `mean(x)`.

```
xyz <- tibble(x = c(10, 20, 30), y = c(20, 40, 60))
xyz |> mutate(across(x:y, mean))
```

```
## # A tibble: 3 × 2
##       x     y
##   <dbl> <dbl>
## 1    20    40
## 2    20    40
## 3    20    40
```

`summarize(across(...))` collapses to one row per group. `mutate(across(...))` keeps the rows.

---

5. `map()`

`map(.x, .f)` applies `.f` to each element of `.x` and returns a **list**.

```
measurements <- list(
  blood_glucose = c(90, 95, 100),
  age = c(20, 30, 40),
  heart_rate = c(70, 75, 80)
)
map(measurements, mean)
```

```
## $blood_glucose
## [1] 95
##
## $age
## [1] 30
##
## $heart_rate
## [1] 75
```

Each element is handled on its own. The names come along.

A data frame is a list of columns, so `map()` handles one column at a time. `select(where(is.numeric))` drops the factor columns of `iris` first. `sd` of a factor is not a useful number.

```
iris |>
  select(where(is.numeric)) |>
  map(sd)
```

```
## $Sepal.Length
## [1] 0.8280661
##
## $Sepal.Width
## [1] 0.4358663
##
## $Petal.Length
## [1] 1.765298
##
## $Petal.Width
## [1] 0.7622377
```

`map(mtcars, class)` asks for the class of every column. That is a useful first look at a new table. The result is a list because `class()` is allowed to return more than one string. `cut` in `diamonds` is an ordered factor, so `class()` returns `"ordered"` and `"factor"`. `map_chr()` would stop there. Section 8 is the rule.

Extra arguments of `.f` go after the function. They are the same on every element.

```
map_dbl(mtcars, mean, na.rm = TRUE)
```

```
##        mpg        cyl       disp         hp       drat         wt       qsec
##  20.090625   6.187500 230.721875 146.687500   3.596563   3.217250  17.848750
##         vs         am       gear       carb
##   0.437500   0.406250   3.687500   2.812500
```

---

6. Write the function once

Copying a block three times is a good moment to name a function. Calling that function three times is a good moment to use `map()`.

Several Excel files with the same odd layout: skip four rows, take sheet 2, and supply the names. One function holds that layout. This chunk is not run here. There are no workbooks in the notes.

```
library(readxl)

read_weird_excel <- function(file) {
  read_excel(
    file,
    sheet = 2,
    skip = 4,
    col_names = c("id", "value", "group")
  )
}

files <- c("data1.xlsx", "data2.xlsx", "data3.xlsx")
datasets <- map(files, read_weird_excel)
combined <- bind_rows(datasets)
```

`datasets` would be a list of three data frames. `bind_rows()` stacks them.

`keep()` holds the elements where a test is true. `discard()` drops them. Then `map_dbl()` summarizes what remains.

```
iris |>
  keep(is.numeric) |>
  map_dbl(mean)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##     5.843333     3.057333     3.758000     1.199333
```

---

7. Anonymous functions

A function you need once does not need a name. Four ways to pass "mean, and drop missing values." Each call prints the same list.

```
map(measurements, mean, na.rm = TRUE)
map(measurements, function(x) mean(x, na.rm = TRUE))
map(measurements, \(x) mean(x, na.rm = TRUE))
map(measurements, ~ mean(.x, na.rm = TRUE))
```

```
## $blood_glucose
## [1] 95
##
## $age
## [1] 30
##
## $heart_rate
## [1] 75
```

`\(x)` is `function(x)`, from R 4.1 on. Use that in new code. `~` is the older purrr formula. `.x` is the current element. Both still run.

How many distinct values are in each column of `iris`:

```
map_int(iris, \(x) length(unique(x)))
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##           35           23           43           22            3
```

---

8. `map_*()` picks the type

`map()` always returns a list. The typed variants require one value of a fixed type from each element, and they return an atomic vector. A surprise length or type stops the walk, which is the point.

Function	What you get back
`map()`	list
`map_dbl()`	double vector
`map_int()`	integer vector
`map_chr()`	character vector
`map_lgl()`	logical vector

```
map_dbl(measurements, mean)
map_chr(mtcars, class)
map_lgl(mtcars, \(x) any(is.na(x)))
map_int(iris, \(x) sum(is.na(x)))
```

```
## blood_glucose           age    heart_rate
##            95            30            75
##       mpg       cyl      disp        hp      drat        wt      qsec        vs
## "numeric" "numeric" "numeric" "numeric" "numeric" "numeric" "numeric" "numeric"
##        am      gear      carb
## "numeric" "numeric" "numeric"
##   mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
## FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##            0            0            0            0            0
```

`class()` on an ordered factor has length 2, so `map_chr(diamonds, class)` stops at `cut`. Use `map(diamonds, class)` when a column may have more than one class.

---

9. `map2()` and `pmap()`

`map2(.x, .y, .f)` walks two objects in step. Element 1 of `.x` is paired with element 1 of `.y`.

```
map2_dbl(c(1, 2, 3), c(10, 20, 30), \(a, b) a + b)
```

```
## [1] 11 22 33
```

Step	First input	Second input	`a + b`
1	1	10	11
2	2	20	22
3	3	30	33

The same pairing draws one normal sample per mean and standard deviation. `n = 5` is fixed. `set.seed()` makes the draws match this page.

```
set.seed(14)
mu <- c(-10, 0, 10)
sigma <- c(1, 1, 2)
map2(mu, sigma, rnorm, n = 5)
```

```
## [[1]]
## [1] -10.661850  -8.281046  -7.878333  -8.502846 -10.036141
##
## [[2]]
## [1]  1.23194518 -0.06488077  1.06899373 -0.37696531  1.04318309
##
## [[3]]
## [1]  9.234356 10.598843 11.348480  9.414367 10.976107
```

`pmap()` walks a list of argument vectors. Names in the list match arguments of the function.

```
set.seed(14)
args <- list(n = 5, mean = mu, sd = sigma)
pmap(args, rnorm)
```

```
## [[1]]
## [1] -10.661850  -8.281046  -7.878333  -8.502846 -10.036141
##
## [[2]]
## [1]  1.23194518 -0.06488077  1.06899373 -0.37696531  1.04318309
##
## [[3]]
## [1]  9.234356 10.598843 11.348480  9.414367 10.976107
```

`map()` is one input. `map2()` is two. `pmap()` is the general case, including three or more.

---

10. One model per group

`group_modify()` hands each group to a function as a data frame and row-binds what you return. The function takes the group's rows and a one-row tibble of the keys. `group_map()` does the same walk and returns a list instead of one data frame.

```
mtcars |>
  group_by(cyl) |>
  group_modify(\(data, key) {
    fit <- lm(mpg ~ wt, data = data)
    tibble(term = names(coef(fit)), estimate = unname(coef(fit)))
  })
```

```
## # A tibble: 6 × 3
## # Groups:   cyl [3]
##     cyl term        estimate
##   <dbl> <chr>          <dbl>
## 1     4 (Intercept)    39.6
## 2     4 wt             -5.65
## 3     6 (Intercept)    28.4
## 4     6 wt             -2.78
## 5     8 (Intercept)    23.9
## 6     8 wt             -2.19
```

`split()` plus `map()` is the same idea without `group_by()`. A string as `.f` pulls that element out of each result. `summary(fit)$r.squared` is how `"r.squared"` finds the number.

```
mtcars |>
  split(mtcars$cyl) |>
  map(\(data) lm(mpg ~ wt, data = data)) |>
  map(summary) |>
  map_dbl("r.squared")
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

`group_walk()` is the grouped form of `walk()`: one side effect per group, nothing collected.

---

11. `walk()`, `imap()`, `map_if()`, `map_at()`

Function	Job
`pmap()`	three or more inputs, or a whole set of arguments
`walk()`	a side effect, such as writing a file
`imap()`	the element and its name or index
`map_if()`	apply `.f` only where a test is true
`map_at()`	apply `.f` at selected names or positions
`group_walk()`	a side effect, one group at a time

`walk()` runs `.f` and invisibly returns `.x`. Use it when you want the plots on disk and you do not need a list of ggplot objects afterwards. This chunk is not run here. It would write three files under `figures/`.

```
dir.create("figures", showWarnings = FALSE)

variables <- c("mpg", "wt", "hp")
walk(variables, \(v) {
  p <- ggplot(mtcars, aes(x = .data[[v]])) +
    geom_histogram(bins = 15)
  ggsave(filename = paste0("figures/", v, ".png"), plot = p)
})
```

`imap()` passes the name as the second argument. `.y` in the formula form is that name.

```
imap_chr(mtcars[c("mpg", "wt")], \(x, name) paste(name, round(mean(x), 1)))
```

```
##        mpg         wt
## "mpg 20.1"   "wt 3.2"
```

`map_if()` leaves the elements that fail the test untouched. `map_at()` touches only the names you list.

```
meas <- list(a = c(1, 2, 3), b = "hi", c = c(10, 20))
map_if(meas, is.numeric, mean)
map_at(meas, "a", \(x) x * 10)
```

```
## $a
## [1] 2
##
## $b
## [1] "hi"
##
## $c
## [1] 15
## $a
## [1] 10 20 30
##
## $b
## [1] "hi"
##
## $c
## [1] 10 20
```

---

12. `map()` beside a for-loop

```
x <- list(1:3, 4:6, 7:9)
map(x, mean)

y <- vector("list", length(x))
for (i in seq_along(x)) {
  y[[i]] <- mean(x[[i]])
}
y
```

```
## [[1]]
## [1] 2
##
## [[2]]
## [1] 5
##
## [[3]]
## [1] 8
## [[1]]
## [1] 2
##
## [[2]]
## [1] 5
##
## [[3]]
## [1] 8
```

Same three means. `map()` builds the output list. The loop makes you name the index and the container.

Base R has the same jobs under other names. purrr's names say what you get back.

purrr	**Base R**
`map()`	`lapply()`
`map_dbl()` and the other typed forms	`vapply()`
`map2()`, `pmap()`	`mapply()`
`map()` when you simplify by hand	`sapply()`, and the type can change

A for-loop is still the right shape for `while`, for an early `break`, and for filling a structure whose next value depends on the previous one, as in the cumulative sum in section 3.

---

13. Which function

You want	Use
The same function on several columns, inside a pipeline	`across()`
One input, a list back	`map()`
One input, a double, integer, character, or logical vector back	`map_dbl()`, `map_int()`, `map_chr()`, `map_lgl()`
Two inputs, paired by position	`map2()`
Three or more inputs	`pmap()`
One result per group, stacked	`group_modify()`, or `split()` then `map()`
A file or a plot per element	`walk()`, or `group_walk()`
The name or the index as well as the element	`imap()`
The function only on some elements	`map_if()`, `map_at()`

The operation is the function. The elements are `.x`. Keeping those two apart is what makes the code shorter the second time you need it.

---

14. Questions

1. What is the difference between `x[1]` and `x[[1]]` when `x` is a list?
2. Why can `map()` walk a data frame?
3. What is the difference between `map()` and `map_dbl()`?
4. How do you take the median of every numeric column?
5. How do you count missing values in every column?
6. What is the difference between `function(x)` and `\(x)`?
7. When do you use `map2()` instead of `map()`?
8. What is the difference between `group_map()` and `group_modify()`?
9. Why is `walk()` the function that saves a folder of plots?
10. How does the for-loop in section 12 become `map()`?

Answers

1. `x[1]` is a list of length 1. `x[[1]]` is the element that was inside.
2. A data frame is a list of columns. `map()` applies `.f` to each column.

3. `map()` returns a list. `map_dbl()` returns a double vector, and it stops if any element is not one double.
4. `df |> summarize(across(where(is.numeric), median))`, or `map_dbl()` after `keep(is.numeric)`.
5. `map_int(df, \(x) sum(is.na(x)))`.
6. They are the same function syntax. `\(x)` is the short form from R 4.1 on.
7. The function needs two values that change together, paired by position.
8. Both walk the groups. `group_map()` returns a list. `group_modify()` row-binds the data frames you return.
9. The point of that walk is the files. `walk()` runs the function for the side effect. `map()` would also build a list you do not need.
10. `map(x, mean)`. The index and the output list are inside `map()`.

---

15. Practice

1. The first two Fibonacci numbers are 0 and 1. Each later number is the sum of the previous two. Use a loop to compute the first 100. Sanity check: `log2()` of the 100th number is about 67.57.
2. `map_dbl()` means of every column of `mtcars`.
3. Number of unique values in each column of `iris`. Repeat with `across()`. `typeof()` of each column of `nycflights13::flights` with `map_chr()`, then with `across()`.
4. p-value of `t.test(mpg ~ am)` within each `cyl` group of `mtcars`.
5. In three lines, keep the `mtcars` columns whose mean is greater than 10 and report those means (`keep()` then `map_dbl()`). You should get four variables.
6. Standard deviation of each numeric column of `iris` with a for-loop, then with `across()`.